

# *Memory Management*

# Memory Management

- Why we need Virtual Memory or Goals of Memory Virtualization?
  - *Security*
  - *Fragmentation*
  - *Insufficient Memory*
  - *Solutions*
- Logical Address & Virtual Address
  - *Address Translation*
  - *Memory Management Unit*
- Address Translation Mechanisms
  - *Segmentation (Obsolete, replaced by paging)*
  - *Swapping (replaced by paging)*
  - *Paging (Modern Memory Management)*
    - Demand Paging

Note: Without memory management, programs would crash into each other, overwrite data, or completely run out of space.

## *Memory Management: Core Problem*

- In the earliest computers, a single program had the entire machine to itself.
- There was no "memory management" because there was nothing to manage - one program, one memory, no sharing.
- Modern systems run many processes concurrently

“Without memory management, every program would need to know exactly where in physical RAM it will run - and no two programs could safely coexist.”

# *Memory Management: Fragmentation*

1 GB

200 MB  
Running

300 MB  
Running

250 MB  
Running

274 MB

1 GB

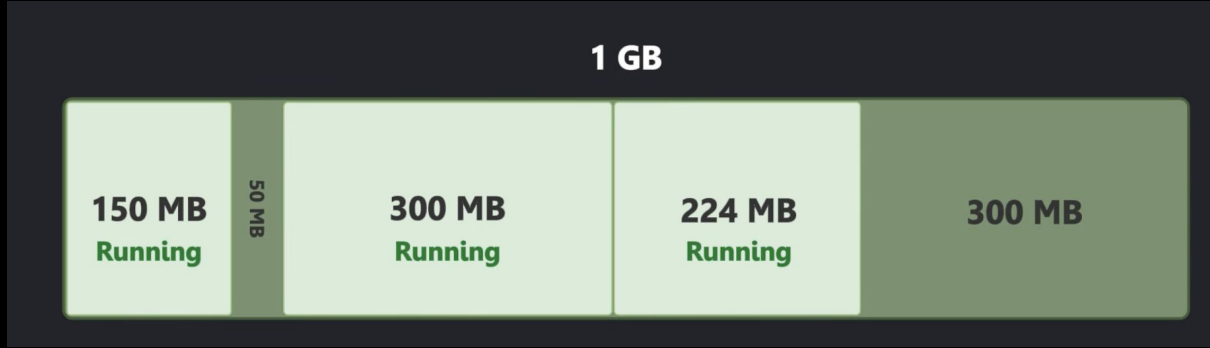
200 MB

300 MB  
Running

250 MB  
Running

274 MB

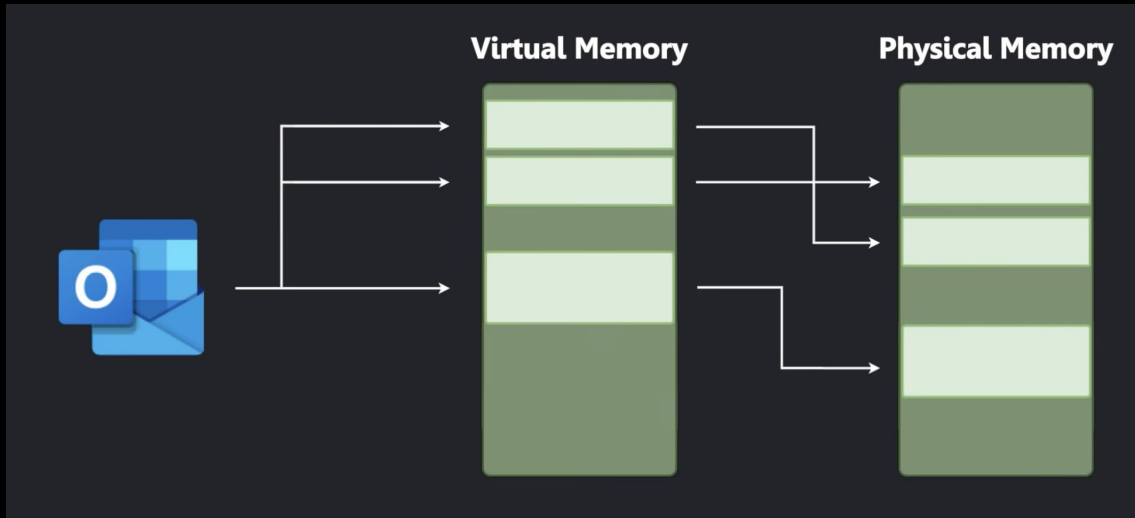
# *Memory Management: Fragmentation*



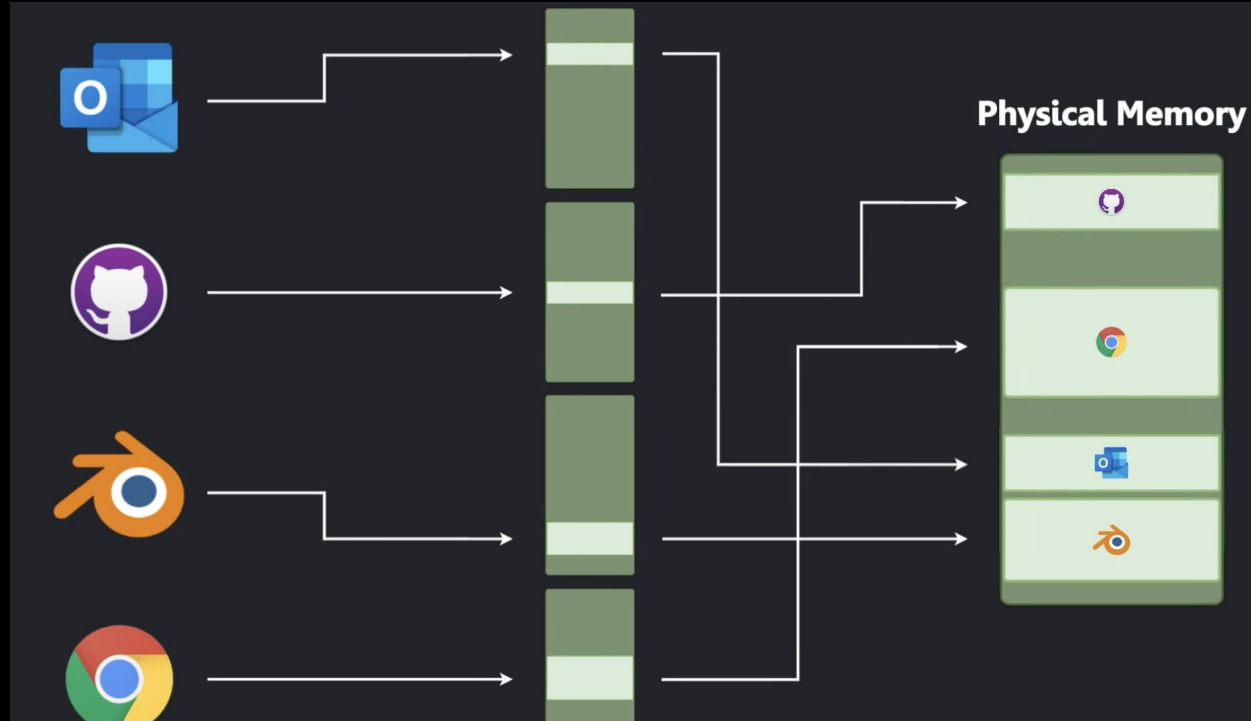
# Memory Virtualization: Single Process

What is Memory Virtualization?

- A technique that **abstracts physical RAM into virtual address spaces** so each process believes it has its own large, continuous block of memory.



# *Memory Virtualization: Multiple Process*



# *Memory Virtualization*

- Programmers were manually splitting their programs into chunks to fit into limited core memory, and manually managing "overlays" — swapping chunks in and out by hand.
- The illusion of a large memory space, with hardware and the OS silently handling the actual movement of data between fast core memory

“Virtual memory wasn't invented for elegance — it was invented because programmers were manually doing the OS's job, splitting programs by hand to fit into tiny physical memories. Automating that illusion is where virtual memory was born.”

# *Memory Virtualization*

"Programmers think they own all of RAM."

A programmer writes code assuming exclusive access to a large, contiguous, private address space — not assuming that all of it is simultaneously backed by physical RAM.

Virtual memory gives each process three illusions simultaneously:

- (1) that it has the entire address space to itself,
- (2) that its memory is contiguous, and
- (3) that it has more memory available than physically exists in the machine.

# *Memory Virtualization*

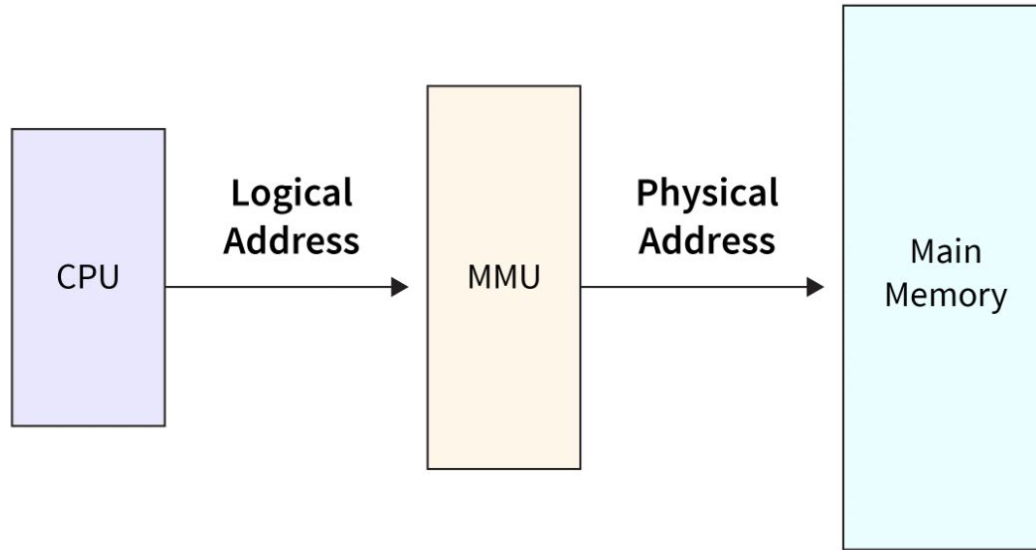
Summary: Why do we need to Virtualize Memory?

1. Avoid Manual Handling of Memory by Developers
2. Larger-Than-RAM Capacity: 8 GB RAM & 12 GB Game
3. Simplified, Contiguous Memory
4. Memory Protection and Permissions

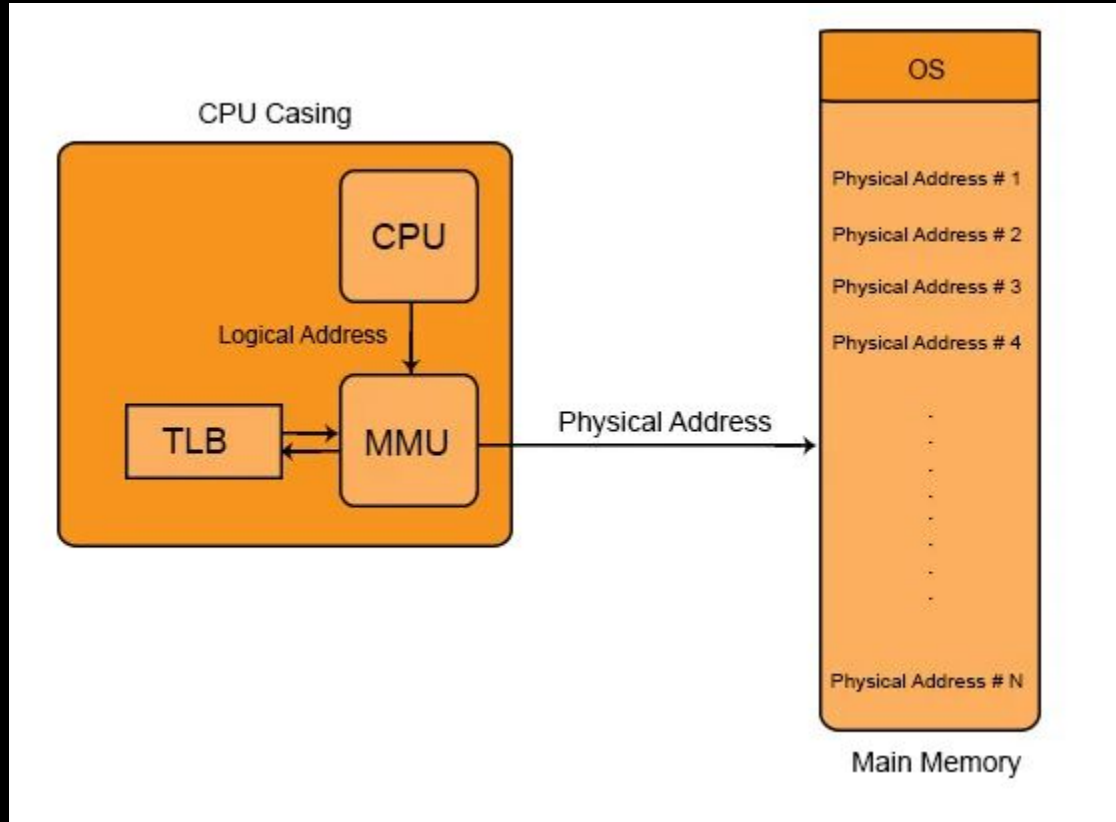
# *Logical and Physical Address*

- In the world of computer operating systems (OS), handling memory is really important.
- One important idea in memory management is dividing memory into two types:
  - *logical and*
  - *physical address spaces.*
- Logical address
- Physical address
- Memory Manager

# *Logical and Physical Address*



# MMU



# *Logical Address*

Logical Address (Virtual Address):

1. Generated by the CPU during program execution.
2. Represents a location in memory from the process's perspective.
3. Unique for each running process.
4. Used for accessing data and instructions.

# *Physical Address*

Physical Address:

1. Represents the actual location in memory hardware.
2. Fixed and does not change during program execution.
3. Used for direct data retrieval from RAM or storage.

# *Logical and Physical Address*

Memory Manager:

1. In charge of converting logical addresses to physical addresses.
2. Ensures proper isolation and memory allocation for running processes.

Multitasking:

1. Logical addresses enable multitasking by allowing multiple processes to run concurrently.
2. Each process has its own set of logical addresses.

# *Logical Address vs Physical Address: Differences*

| Difference     | Logical Address                                                                                       | Physical Address                                                                                         |
|----------------|-------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Definition     | Generated by the CPU during program execution, representing a process's view of memory.               | Represents the actual location in memory hardware, pointing directly to data in hardware memory.         |
| Responsibility | Managed by the CPU and generated for each running process, ensuring process isolation.                | Managed by the hardware and remains static during program execution, providing stable memory addressing. |
| Translation    | Requires translation by the memory manager or hardware before accessing data; it's a virtual address. | Does not require translation; it directly represents the data's physical location in memory.             |
| Uniqueness     | Unique for each running process, allowing processes to run independently.                             | Unique for each storage cell in memory, providing precise addressing within hardware memory.             |

# *Logical and Physical Address*

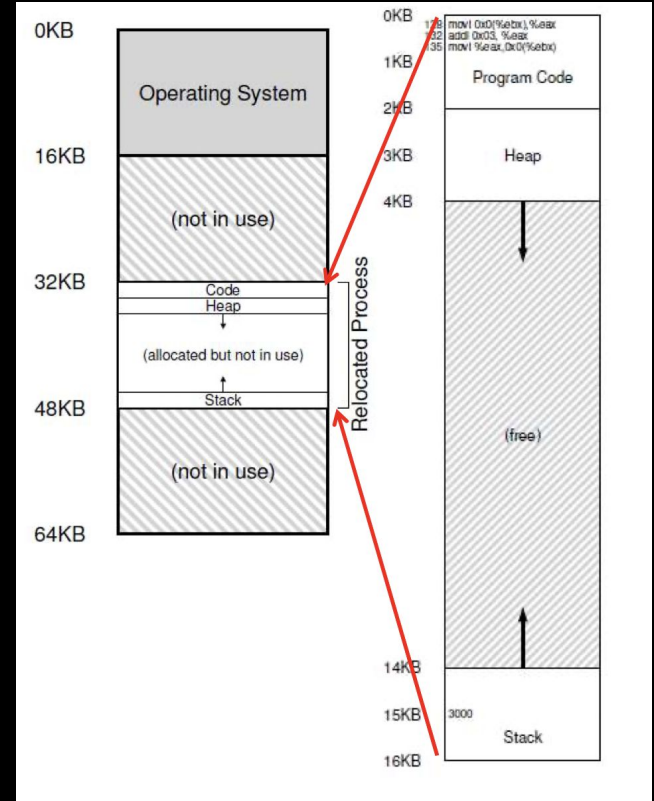
Walk through the sequence explicitly:

1. CPU fetches an instruction, which references logical address 346.
2. This logical address is sent to the MMU — the CPU itself never touches physical memory directly.
3. The MMU applies a translation rule (in this session: base + limit) to compute the physical address.
4. If the address is invalid (out of bounds), the MMU raises a hardware trap, and the OS handles it (typically terminating the process with a segmentation fault).
5. If valid, the physical address goes out on the memory bus and the data/instruction comes back.

# *How are virtual address spaces mapped to physical memory?*

Simplest form of memory management: **base and bound**

- Place memory image  $[0, N]$  contiguously starting at memory address base  $B$
- Virtual address  $X$  translated to physical address  $B + X$
- Access to virtual addresses beyond  $N$  will not be permitted
- OS provides base and bound to MMU for translation/error checking
- When CPU access a VA, MMU computes  $PA = VA + \text{base}$ , physical memory accessed with  $PA$



## *Role of OS vs MMU*

- OS allocates memory, builds translation information of process
  - *But OS does not do the actual address translation on every memory access*
  - *Why? Once user code starts running on CPU, OS is out of the picture (until a trap)*
  - *When process is switched in, translation information is provided to MMU*
- CPU runs process code, accesses code/data at virtual addresses
  - *Virtual addresses translated to physical addresses by MMU*
  - *Actual physical memory is accessed using physical addresses*
- MMU raises a trap if there is any error in the address translation
  - *CPU executes trap instruction, OS code runs to handle the error*
- OS gives new information to MMU on every context switch

The CPU understands Virtual Memory Addresses.

The OS allocates Physical Memory Addresses.

## *PYQs*

Which of the following is the primary purpose of virtual memory?

- A. Increase CPU clock speed
- B. Allow execution of processes larger than physical memory
- C. Increase cache size
- D. Eliminate page faults

*PYQ*

Which of the following is the primary purpose of virtual memory?

- A. Increase CPU clock speed
- B. Allow execution of processes larger than physical memory
- C. Increase cache size
- D. Eliminate page faults

Answer:

- B. Allow execution of processes larger than physical memory

Virtual memory gives a process the abstraction of a large address space even when physical RAM is smaller.

## *Practice*

A1. A process has base = 2400 and bound = 670. It issues virtual address 410. What is the physical address, and is the access valid?

1. 2810; valid
2. 2810; invalid
3. 1990; valid
4. 3070; valid

## *Practice*

A1. A process has base = 2400 and bound = 670. It issues virtual address 410. What is the physical address, and is the access valid?

1. 2810; valid
2. 2810; invalid
3. 1990; valid
4. 3070; valid

1.  $410 < 670$ , so valid;  $PA = 2400 + 410 = 2810$ . Option 3 subtracts instead of adding; option 4 adds the bound instead of the VA.

## *Practice*

A2. A process has base = 5600 and bound = 940. It issues virtual address 1260. What happens?

1. Physical address 6860 is accessed
2. Physical address 4340 is accessed
3. The MMU raises a trap and the OS handles the fault
4. The address is clipped to 940, so physical address 6540 is accessed

## *Practice*

A2. A process has base = 5600 and bound = 940. It issues virtual address 1260. What happens?

1. Physical address 6860 is accessed
  2. Physical address 4340 is accessed
  3. The MMU raises a trap and the OS handles the fault
  4. The address is clipped to 940, so physical address 6540 is accessed
3.  $1260 > 940$ , so the MMU traps before any memory is touched. Option 1 is the classic mistake of adding without checking the bound.

## *Practice*

A3. When the OS switches the CPU from process P to process Q, who sets the base and bound registers to Q's values?

1. Process Q itself , as the first instruction it runs in user mode
2. The OS, in kernel mode, using the values saved for Q
3. The MMU, which computes them from Q's PID automatically
4. Nobody; all processes share one base and bound

## *Practice*

A3. When the OS switches the CPU from process P to process Q, who sets the base and bound registers to Q's values?

1. Process Q itself , as the first instruction it runs in user mode
  2. The OS, in kernel mode, using the values saved for Q
  3. The MMU, which computes them from Q's PID automatically
  4. Nobody; all processes share one base and bound
2. Only the kernel can load base and bound. It takes Q's values from Q's saved state (PCB) during the context switch.

## *Practice*

A4. Why are the instructions that load the base and bound registers privileged (kernel mode only)?

1. Because they are slower than ordinary instructions
2. Because the compiler must insert them at build time
3. Because user programs never need to know their own addresses
4. Because a user process could otherwise change its own limits and reach other processes' memory

## *Practice*

A4. Why are the instructions that load the base and bound registers privileged (kernel mode only)?

1. Because they are slower than ordinary instructions
  2. Because the compiler must insert them at build time
  3. Because user programs never need to know their own addresses
  4. Because a user process could otherwise change its own limits and reach other processes' memory
4. If a user process could load these registers, it could raise its own bound or move its base and read or overwrite any memory, so protection would be meaningless.

## *Practice*

1. Validity table. A process has base = 1400 and bound = 360. For each virtual address, say whether it is valid and give the physical address if it is: (a) 75 (b) 510 (c) 290 (d) 930.
2. B2. Working backwards. For the same process (base = 1400, bound = 360), which virtual address, if any, produced each physical address? (a) 1630 (b) 1955. Justify each answer.

# *Solutions*

## 1. Solution:

- a.  $75 < 360$  Valid,  $PA = 1400 + 75 = 1475$
- b.  $510 > 360$  Invalid, MMU traps
- c.  $290 < 360$  Valid,  $PA = 1400 + 290 = 1690$
- d.  $930 > 360$  Invalid, MMU traps

## 2. Solution:

- a.  $VA = 1630 - 1400 = 230$ , and  $230 < 360$ , so VA 230 produced it.
- b.  $1955 - 1400 = 555$ , but  $555 > 360$ . No valid VA of this process maps to 1955: that physical address is outside the process's region, so the process can never reach it.

# *Swapping*

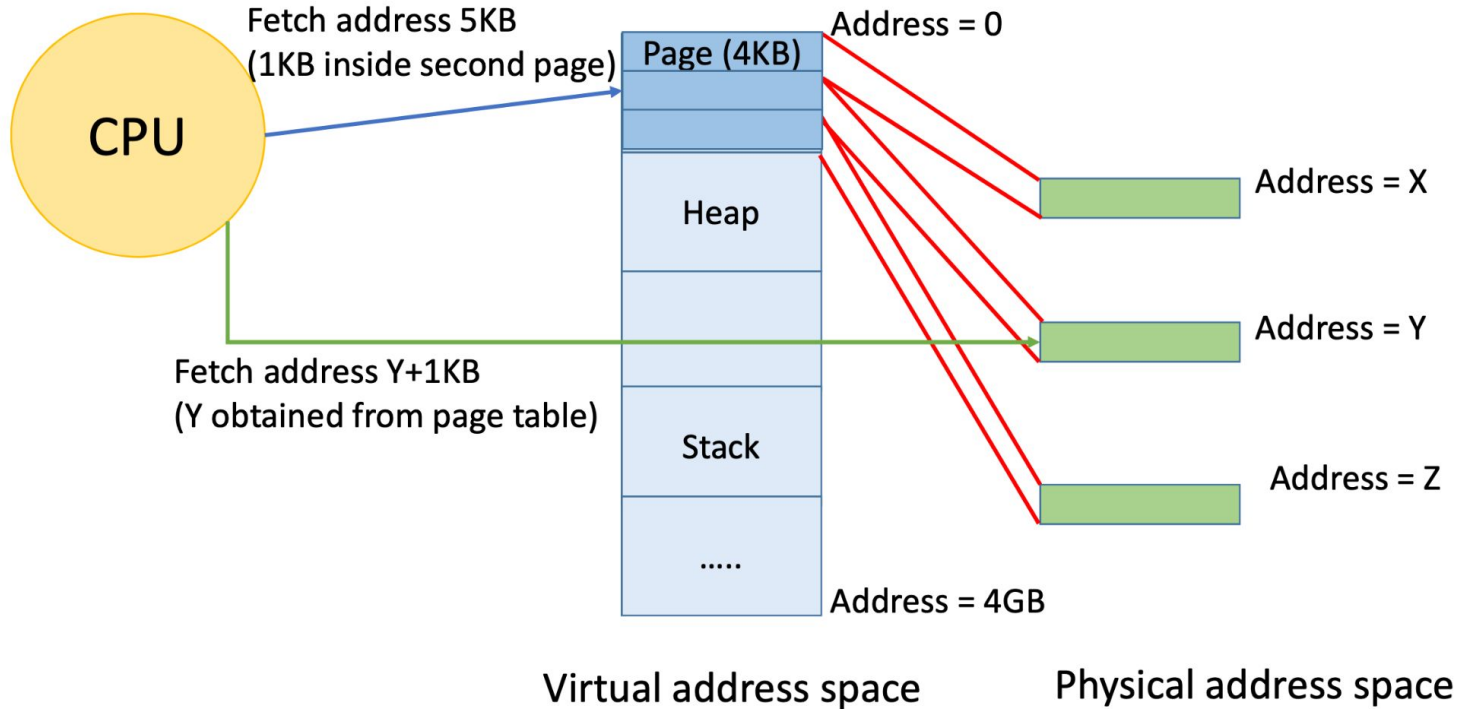
- In classic operating systems, swapping meant moving an entire process (all of its code, stack, data, and heap) out of RAM onto a hard drive to free space
- Modern operating systems no longer swap whole processes.
- Instead, they use a hybrid approach often called paging out (or page swapping). They only kick individual, inactive pages out to the storage drive (into a pagefile or swap space) while keeping the active pieces of the process running in RAM.

*Paging*

# Paging

- Widely used memory management system today
- Virtual address space divided into **fixed size pages**
- Each page is assigned a free physical frame by OS
- Memory allocation is at granularity of fixed size pages (e.g., 4KB)
- Why paging? Avoids **external fragmentation**
  - *No wastage of space due to gaps between allocated and free memory*
  - *Internal fragmentation may be there (space wasted inside partially filled page)*
- Disadvantage: **internal fragmentation** (partially filled pages)
- Page table of a process maps **virtual page numbers (VPN)** to **physical frame numbers (PFN)** for every page

# Paging

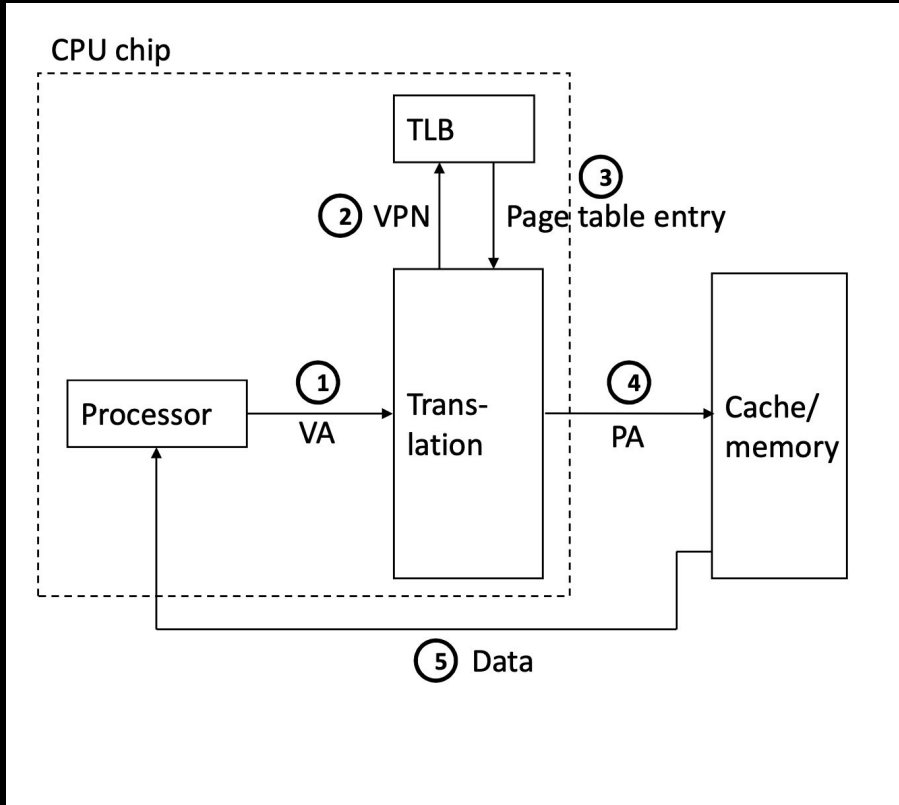


# *Translation Lookaside Buffer (TLB)*

- Overhead of memory translation: every memory access preceded by extra memory accesses to read page table
- To reduce this overhead, MMU caches the most recent translations in translation lookaside buffer (TLB)
- TLB only caches page table entries (VPN PFN mappings), not actual memory contents
  - *Different from CPU caches that cache actual memory contents*
- If TLB hit, fetch memory contents in one memory access
- If TLB miss, MMU must perform extra memory access for page table access (“page table walk”)
- TLB flush on context switch: mappings cached in TLB change

## *TLB hit*

1. CPU accesses virtual address
2. MMU looks up page number in TLB
3. If TLB hit, page table entry is available, physical address computed
4. CPU directly accesses required code/data using physical address



## *TLB miss*

1. CPU accesses virtual address
2. MMU looks up page number in TLB, cannot find entry
3. MMU looks up page table in memory to find page table entry
4. Page table entry populated in TLB for future use
5. MMU computes physical address using which CPU accesses main memory

